CST2550 Software Engineering Management and Development
TV Programme Scheduling System
Group Coursework - Report
Submission Date: 10th April 2026

Team Leader (Scrum Master)	Khrisvin	Nepaulsing	M01003462
Secretary	Veedoorsingh	Gopal	M01013795
Developer	Mehdi	Bhoyroo	M00983496
Developer	Navneet	Sookraz	M01000843
Tester	Pranay	Poheith	M01013276

Contents

1. Introduction 1

1.1 Project Overview 1

1.2 Report Structure 1

2. Design 1

2.1 System Overview / Architecture 1

Figure 1. High-level architecture of the TV Programme Scheduling System 2

2.2 Data Structure Selection and Justification 2

2.3 Algorithm Design Using Pseudocode 3

2.4 Database Design 4

Figure 2. Simplified entity-relationship diagram of the implemented SQL database schema. 5

3. Testing 5

3.1 Testing Approach 5

3.2 Test Cases Table 5

4. AI Agent 6

4.1 AI Agent Design 6

Figure 3. Simplified use case diagram of the AI-inspired scheduling assistant for administrative users 6

4.2 Implementation Overview 7

5. Conclusion 7

5.1 Summary of Work 7

5.2 Limitations 7

5.3 Critical Reflection 7

6. References 8

7. Appendix A: Test Evidence 8

Figure A1. Failed tests screenshot 8

Figure A2. Passed tests screenshot 9

1. Introduction

1.1 Project Overview

The TV programme scheduling system is a desktop application developed in C# using Windows Forms. It is designed to support efficient schedule management through functions such as adding, removing, and viewing schedules by channel, as well as suggesting suitable time slots. The system addresses a practical scheduling problem in which manual planning can lead to overlaps, poor time allocation, and disorganisation.

The system supports four user roles: Admin, Manager, Client, and Viewer. Admin has full control, while Manager has similar scheduling access, except for schedule removal. Client has more limited access, and Viewer has similar access to Client but is restricted by a timed session.

The system also supports programme management through a pre-populated programme list and an Add Program feature, allowing new programmes and preview images to be stored in the database. A custom hash table was implemented as the primary structure for schedule storage, while the system also includes an integrated AI assistant that can respond to schedule-related queries using the current schedule context.

1.2 Report Structure

This report outlines the system design, key scheduling algorithms, testing approach, AI-inspired assistant, and a concluding reflection on limitations and future improvements.

2. Design

2.1 System Overview / Architecture

The system was designed using a layered structure so that the user interface, scheduling logic, storage, database access, and supporting data models each had a clear responsibility. This improves organisation and reduces the mixing of interface code, business rules, and data handling.

The main interface is provided through forms such as MainForm, LoginForm, ClientForm, and ViewerForm, which support role-based access and user interaction. The service layer includes ScheduleManager, UserService, and AIService, while data access is handled through repositories such as ScheduleRepository, ProgramRepository, and UserRepository.

Schedules are stored in ScheduleStorage, which implements a custom hash table using generated keys based on channel and start time. Supporting model classes such as Schedule, User, and ProgramItem represent the main entities used throughout the system.

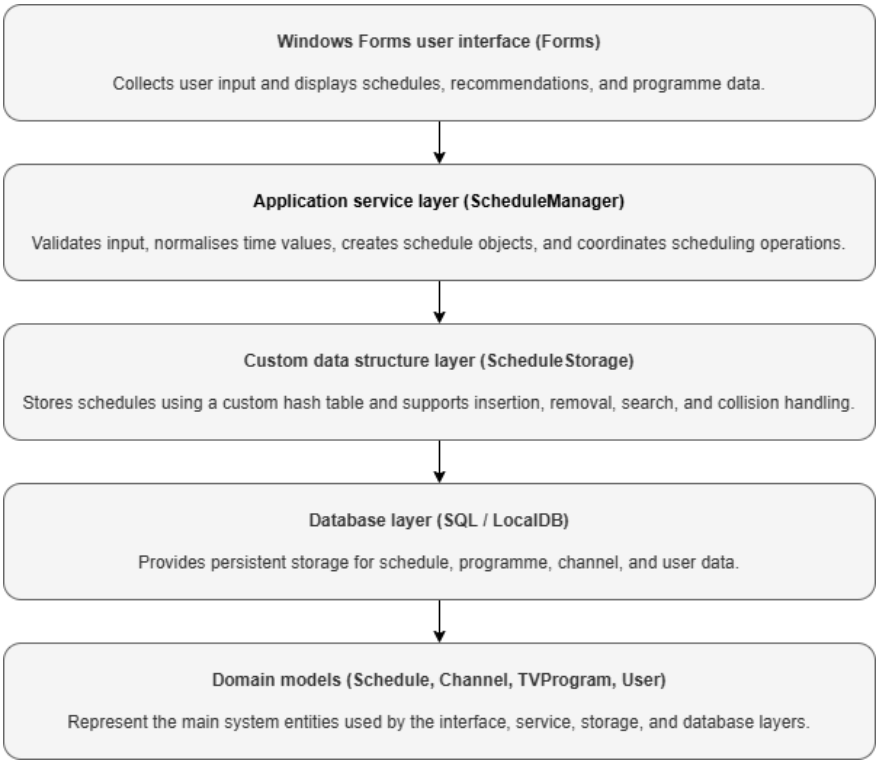

Figure 1. High-level architecture of the TV Programme Scheduling System

This layered arrangement supports the scheduling workflow by allowing user actions to pass through the service and repository layers before schedules and programmes are stored, retrieved, or analysed.

Component	Responsibility	Main benefit
MainForm / Forms	Handles user interaction and displays schedules	Separates UI from logic
ScheduleManager	Validates input and manages scheduling operations	Centralises business logic
ScheduleStorage	Stores and retrieves schedules using a custom hash table	Efficient key-based access
Model Classes	Represent system entities such as schedules and channels	Improves structure and clarity
AI Assistant Logic	Suggests slots, channels, and conflict explanations	Supports decision-making

2.2 Data Structure Selection and Justification

A major design decision in this project was the choice of a custom hash table as the primary structure for storing schedules. A hash table stores and retrieves records through keys mapped to indices by a hash function. In this system, each schedule is identified by a key based on channel ID and start time.

This structure was chosen because the system frequently performs operations that depend on fast access, including adding schedules, checking duplicates or conflicts, removing entries, and retrieving schedules for display. To handle collisions, the implementation uses separate chaining, allowing multiple schedule records to be stored in the same bucket without data loss.

Compared with arrays or lists, a hash table avoids full linear scans for direct key-based operations. Compared with balanced trees such as AVL trees, it is more suitable here because the system relies mainly on direct access rather than continuous ordered traversal. This makes the custom hash table well aligned with the operational needs of the scheduling system.

Operation	Hash Table	Simple List	Explanation
Search by Key	O(1) average	O(n)	Direct lookup using generated key
Insert	O(1) average	O(n)	No need to scan or shift items
Delete	O(1) average	O(n)	Item can be removed directly from bucket
Traverse All	O(n)	O(n)	Full scan still required

This comparison shows that the custom hash table offers the best fit for the system's need for fast key-based schedule operations while remaining simpler than a balanced tree.

2.3 Algorithm Design Using Pseudocode

The following pseudocode summarises the main scheduling operations in a language-independent form rather than repeating the original C# code.

These algorithms reflect the main system behaviours. Add, Remove, and Search Schedule rely on the custom hash table for efficient key-based access. Suggest Next Available Slot identifies the earliest valid gap, while the final algorithm outlines the rule-based recommendation logic behind the AI-inspired administrative assistant.

Add Schedule:

```
INPUT channelId, programId, startTime, duration

IF duration <= 0 THEN
   RETURN error
END IF

normalisedStartTime ← round startTime to nearest minute
endTime ← normalisedStartTime + duration

newSchedule ← create Schedule(channelId, programId, normalisedStartTime, endTime)

key ← createKey(channelId, normalisedStartTime)
index ← hash(key)

FOR each schedule IN bucket[index]
   IF schedule overlaps with newSchedule THEN
      RETURN false
   END IF
END FOR

INSERT newSchedule into bucket[index]
RETURN true
```

Remove Schedule:

```
INPUT channelId, startTime

key ← createKey(channelId, startTime)
index ← hash(key)

FOR each schedule IN bucket[index]
   IF schedule.channelId = channelId AND schedule.startTime = startTime THEN
      REMOVE schedule from bucket[index]
      RETURN true
   END IF
END FOR

RETURN false
```

Search Schedule:

```
INPUT channelId, startTime

key ← createKey(channelId, startTime)
index ← hash(key)

FOR each schedule IN bucket[index]
   IF schedule.channelId = channelId AND schedule.startTime = startTime THEN
      RETURN schedule
   END IF
END FOR

RETURN null
```

Suggest Next Available Slot:

```
INPUT channelId, requiredDuration

currentTime ← current system time
scheduleList ← get all schedules for channelId

IF scheduleList is empty THEN
   RETURN currentTime
END IF

SORT scheduleList by startTime

FOR each schedule IN scheduleList
   IF currentTime + requiredDuration <= schedule.startTime THEN
  RETURN currentTime
   ELSE
      currentTime ← schedule.endTime
   END IF
END FOR

RETURN currentTime
```

Recommend Best Channel:

```
INPUT preferredStartTime, duration

bestChannel ← null
bestTime ← null

FOR each channel
   candidateTime ← find earliest valid slot for channel from preferredStartTime

   IF bestTime is null OR candidateTime is earlier than bestTime THEN
      bestTime ← candidateTime
      bestChannel ← channel
   END IF
END FOR

RETURN bestChannel
```

Together, these algorithms show how the system combines efficient schedule operations with rule-based decision support for administrative users.

2.4 Database Design

The system uses an SQL database to provide persistent storage for programme, schedule, and user data. This allows records to remain available after the application is closed and reopened.

The implemented database is centred on three main tables: Programs, Schedule, and Users. The Programs table stores programme codes, names, and image paths, the Schedule table stores each broadcast with its channel, programme identifier, start time, end time, and optional image path, and the Users table stores login credentials and roles for database-backed accounts.

Table	Key Fields	Purpose	Note
Programs	ProgramCode (PK), ProgramName, ImagePath	Stores programme details used in the programme dropdown and preview display	Supports programme management and Add Program
Schedule	ScheduleID (PK), ChannelID, ProgramID, StartTime, EndTime, ImagePath	Stores each scheduled broadcast	Linked to Programs through ProgramID
Users	UserID (PK), Username, Password, Role	Stores login and role data for database-backed accounts	Supports client registration and login

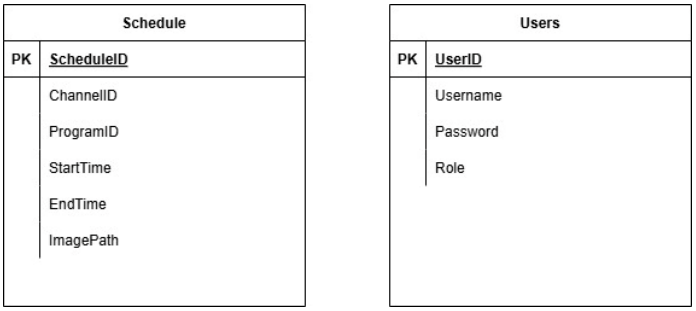

Figure 2. Simplified entity-relationship diagram of the implemented SQL database schema.

The SQL database provides persistent storage for confirmed programme, schedule, and user records, while the custom hash table supports efficient in-memory schedule handling. When a scheduling action is performed through the interface, the request is passed to the service layer, which validates it and coordinates the necessary storage or database operation.

This combination creates a balanced design in which the custom data structure supports fast schedule processing, while SQL ensures durable and structured storage beyond the current runtime session.

3. Testing

3.1 Testing Approach

The project repository includes an MSTest project with automated tests for four key behaviours: adding a schedule without conflict, rejecting an overlapping schedule, removing an existing schedule, and retrieving a schedule by channel and start time. MSTest provides a standard framework for defining test classes, test methods, and assertions in .NET applications (Microsoft, 2026a; Microsoft, 2026b).

This testing approach is appropriate because the main risks are logical rather than visual, particularly overlap handling, deletion accuracy, and schedule retrieval. Additional manual and edge-case testing would further strengthen confidence in areas such as programme insertion, channel-specific conflicts, duplicate start times on different channels, database persistence scenarios, and AI assistant behaviour.

3.2 Test Cases Table

Table 1 combines the automated tests already present in the repository with additional recommended checks that would further strengthen quality assurance.

Test case	Input	Expected output	Actual output / status	Type	Result
TC1 Add valid schedule	Channel 1, 10:00, 60 mins	Method returns True.	Implemented in MSTest suite; expected behaviour asserted.	Valid / automated	Pass
TC2 Add conflicting schedule	Second item starts at 10:30 while first ends at 11:00	Method returns False.	Implemented in MSTest suite; overlap correctly rejected.	Invalid / automated	Pass
TC3 Remove existing schedule	Channel 1, start time 10:00	Method returns True.	Implemented in MSTest suite; removal asserted.	Valid / automated	Pass

TC4 Retrieve existing schedule	Channel 1, start time 10:00	Non-null schedule with ProgramID 100.	Implemented in MSTest suite; retrieved value asserted.	Valid / automated	Pass
TC5 Remove missing schedule	Channel 1, start time not stored	Method returns False.	Not implemented as a test in the repository; recommended additional case.	Non-existent item	Not yet executed
TC6 Invalid duration	Duration = 0 or negative	ArgumentException raised.	Not implemented as a test in the repository; behaviour follows service validation.	Invalid input	Not yet executed
TC7 Suggest slot in empty channel	No schedules exist for chosen channel	Current time suggested.	Inferred from UI code path in MainForm.	Edge case / manual	Not yet executed
TC8 Suggest slot where no gap exists	Schedules fully occupy the current timeline	End time of last schedule suggested.	Inferred from UI code path in MainForm.	Edge case / manual	Not yet executed

4. AI Agent

4.1 AI Agent Design

The system includes an integrated AI assistant to support administrative decision-making. The assistant combines internal scheduling logic with external AI chat support. Rather than relying only on fixed interface rules, it can also respond to user questions using the current schedule context, including the selected channel, requested start time, duration, and the schedules already stored in the system.

The assistant supports three main functions: slot suggestion, conflict explanation, and channel or timing guidance. Within the interface, the scheduling logic can suggest the next available slot and help avoid overlaps. In addition, the AI chat assistant can respond to user queries about whether a slot is valid, whether a conflict exists, and whether a better time or channel should be considered.

Overall, the assistant acts as a decision-support feature rather than a fully autonomous scheduler. It improves the usability of the system by combining direct schedule analysis with contextual AI responses.

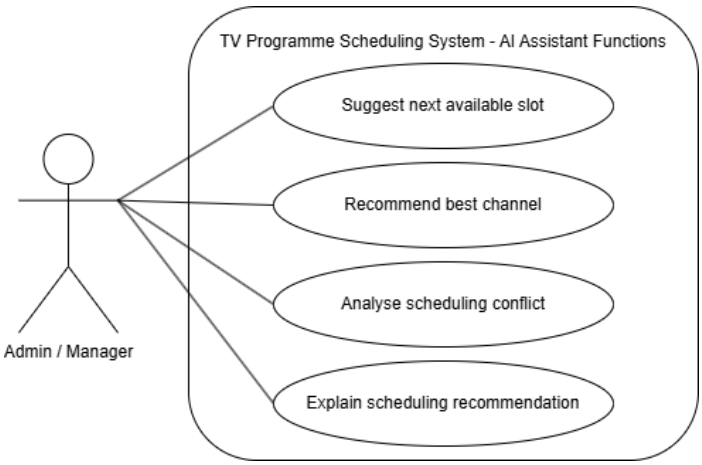

Figure 3. Simplified use case diagram of the AI-inspired scheduling assistant for administrative users

4.2 Implementation Overview

The AI assistant was implemented as an integrated feature of the administrative workflow rather than as a separate subsystem. In the interface, schedule-related actions such as slot suggestion are handled directly by the application logic, while conversational AI responses are managed through AIService.

The assistant uses the current schedule context, including all stored schedules, the selected channel, requested start time, and duration, and sends this information together with the user's message to an external Gemini model. The returned response is then displayed in the AI panel of the admin interface.

This design keeps the assistant aligned with the overall architecture of the system: the interface handles interaction, the service layer manages logic, the repository and database layers provide persistent data, and the custom structure supports schedule processing.

5. Conclusion

5.1 Summary of Work

The completed system supports the core functions expected of a TV scheduling application, including adding, removing, and viewing schedules by channel, as well as suggesting suitable programme placement. It also includes role-based access control, programme management through a database-backed programme list, SQL persistence, and an integrated AI assistant for schedule-related support.

The project combines a custom hash table for efficient schedule handling, a relational database for durable storage, and both interface-based and conversational AI support within a single working application.

5.2 Limitations

The current assistant is not fully autonomous and depends on the schedule context provided by the application rather than deeper historical or predictive analysis. The custom hash table supports efficient key-based access, but operations that require time ordering still need additional processing such as sorting or scanning. The system could also be improved through stronger security for API key handling, more advanced validation, and broader testing of AI and programme-management features.

5.3 Critical Reflection

If developed further, the system could be improved by extending the assistant beyond context-based guidance, refining the scheduling logic to consider quality of placement as well as conflict avoidance, and broadening testing to cover more edge cases, programme-management workflows, AI responses, role permissions, and database scenarios. Overall, the project provides a strong functional foundation while leaving clear opportunities for future technical improvement.

6. References

lofe2026 (2026) *TV-Schedule-System-CST2550-*. Available at: https://github.com/lofe2026/TV-Schedule-System-CST2550- (Accessed: 2 April 2026).

Microsoft (2026a) *MSTest overview*. Available at: https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-mstest-intro. (Accessed: 2 April 2026).

Microsoft (2026b) *Write tests with MSTest*. Available at: https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-mstest-writing-tests (Accessed: 2 April 2026).

Visual Paradigm (2019) *What is Use Case Diagram?* Available at: https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-use-case-diagram/ (Accessed: 2 April 2026).

7. Appendix A: Test Evidence

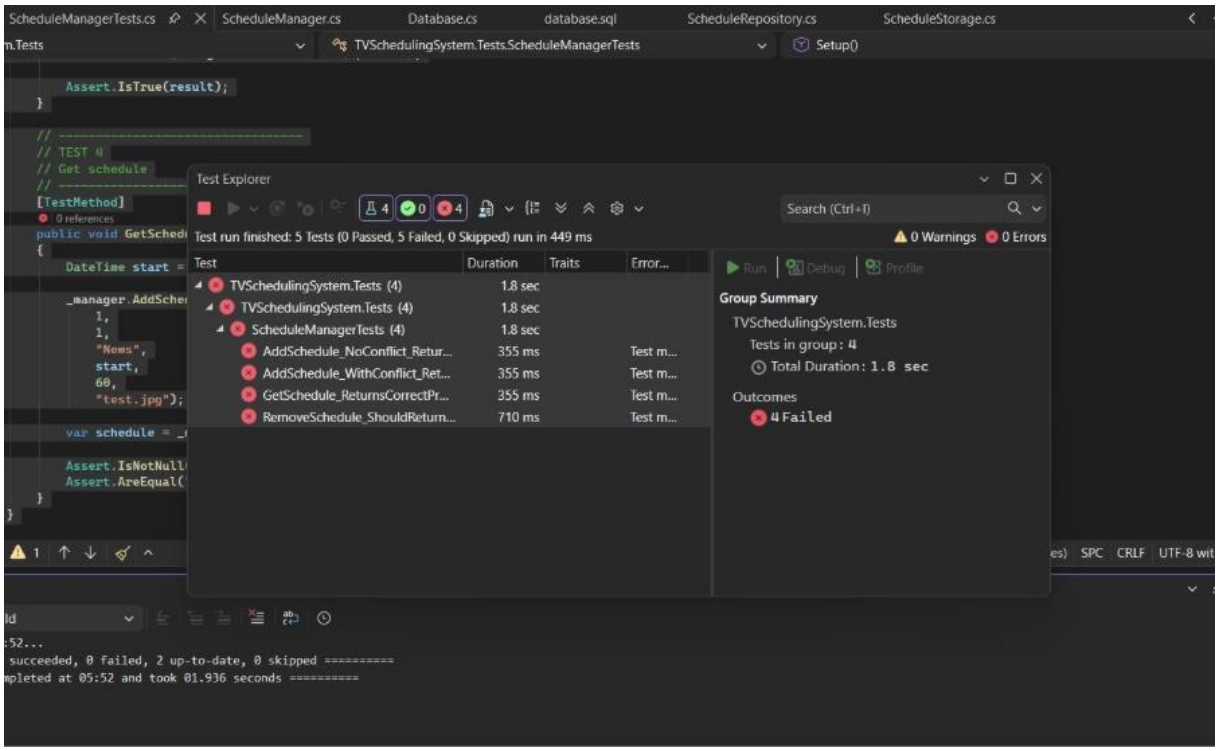

Figure A1. Failed tests screenshot

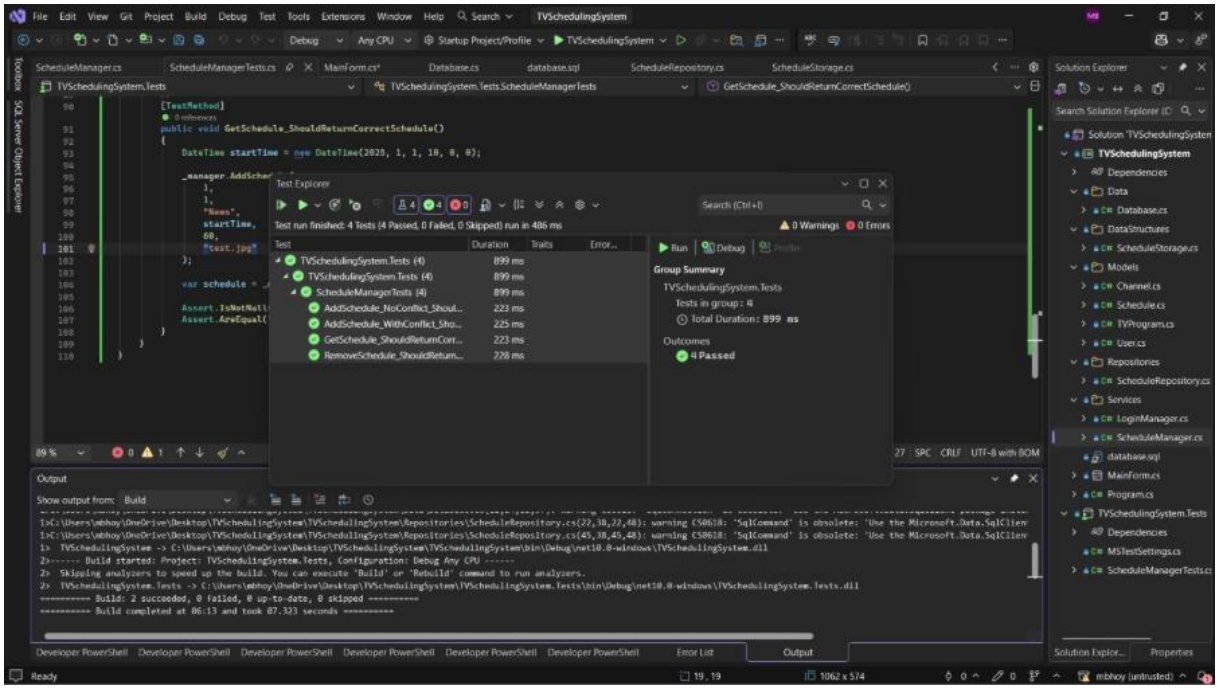

Figure A2. Passed tests screenshot